Example usage of Mega2R

Robert V. Baron and Daniel E. Weeks

June 22, 2017, 10:00

Contents

1 Tutorial directory and files

The files you will need for this tutorial are in the mega2rtutorial directory. You should **cd** there to do these exercises.:

You should see the following files:

[1] "MEGA2.BATCH.seqsimr"	"MEGA2.BATCH.srdta"	"MEGA2.BATCH.vcf"
[4] "Mega2r.map"	"Mega2r.ped"	"mega2r.Rmd"
[7] "mega2r.pdf"	"seqsimr.db"	

2 Installation

Note: The code in this section should only be executed once, to load the packages into R and are NOT run for every R session.

Also, we will assume that you have started an R session at which to type the commands in the tutorial.

To run these exercises, you should install the CRAN packages *mega2r*, *mega2vcf*, *mega2genabel* and *mega2pedgene*. Once they are in CRAN, the preferred way to install will be to use the install.packages() command. For now, we prefer to use devtools in R; as in

```
devtools::install("../mega2r")
devtools::install("../mega2pedgene")
devtools::install("../mega2vcf")
devtools::install("../mega2genabel")
```

Certainly,

R CMD install mega2r
R CMD install mega2pedgene
R CMD install mega2vcf
R CMD install mega2genabel

will work, too. It may generate warnings about non-package related items in the tree. All these packages may fetch other dependent CRAN packages. We also require Bioconductor Annotations that must be loaded manually. The first line loads the Bioconductor loader and the next two lines load two annotation databases. One for gene transcript locations and one to map gene names to the entrez gene IDs. (Note: You may choose a different transcript database from Bioconductor or construct one of your own.) Please type in R:

```
source("https://bioconductor.org/biocLite.R")
biocLite("TxDb.Hsapiens.UCSC.hg19.knownGene")
biocLite("org.Hs.eg.db")
```

3 Example Data

We have chosen to use the SeqSIMLA2 program to generate a data set for us. We needed to sub-sample the data down to 1,380 people and 1,000 markers to make the size manageable. These data will be used to illustrate Mega2 and Mega2R operations that follow.

Note: This simulator only produces markers on chromosome 1.

4 Using Mega2R packages

4.1 Creating a Mega2 database

We have provided files in the mega2rtutorial directory, that contain the data from the simulation. These files are in PLINK ped format data for 1,380 samples with 1,000 markers:

- seqsimr.ped
- seqsimr.map

If you do not wish to install Mega2 right now, you can use the seqsimr.db database that is in the tutorial directory. Of course, the tutorial steps that invoke Mega2 will fail, mainly the regression steps.

You will need to obtain the Mega2 program from https://watson.hgen.pitt.edu/register/. First, you will invoke Mega2 on your data. To make matters simple, we will use a pre-constructed Mega2 batch file to automate the processing by Mega2. To run Mega2 to process and create the SQLite3 database 'seqsimr.db', we issue this command at the Unix prompt:

```
mega2 MEGA2.BATCH.seqsimr
```

If you do not provide the argument, Mega2 will proceed to ask a series of questions to collect the needed information to produce a database. In addition, it will create a Mega2.BATCH file, we've suggested you use. You can look at the "Quick Start" documentation https://watson.hgen.pitt.edu/docs/mega2_html/mega2.html Section 4. to better understand the interactive process.

This MEGA2.BATCH.seqsimr file begins with a rather long comment indicating the keyword values that may be set and their default value. Toward the end of the file, we set the inputs as **Mega2r.ped** and **Mega2r.map**, indicate the input is PLINK ped format with parameters, and indicate that Mega2 should produce a database called **seqsimr.db**, etc. (These particular items are in bold face text below.)

Input_Database_Mode=1
Input_Format_Type=4
Input_Pedigree_File=Mega2r.ped
Input_PLINK_Map_File=Mega2r.map
Output_Path=.
Input_Path=.
PLINK_Args= –cM –missing-phenotype -9 –trait default
Input_Untyped_Ped_Option=2
Input_Do_Error_Sim=no
AlleleFreq_SquaredDev=999999999.000000
Value_Marker_Compression=1
Analysis_Option=Dump
Value_Missing_Quant_On_Input=-9.000000
Value_Missing_Affect_On_Input=-9
Count_Genotypes=4
Count_Halftyped=no
Value_Genetic_Distance_Index=0
Value_Genetic_Distance_SexTypeMap=0
Value_Base_Pair_Position_Index=1
Default_Reset_Invalid=no
DBfile_name=seqsimr.db
Default_Outfile_Names=yes

You will eventually have to run Mega2 to convert your data.

4.2 Reading a Mega2 database into R

After you have the database, start up the R program. The first R commands you should type are:

```
library(mega2r)
ENV = read.Mega2DB("seqsimr.db", verbose = TRUE)
```

The first argument should be the path to the database. Providing the optional argument, verbose, causes the read function to summarize the tables created, their fields and their sizes. Finally, an "R environment" that contains the database tables is returned. (If you are unfamiliar with environments, you can think of them as

data frames. If fact, ENV$locus_table will access the locus_table variable from ENV as though fetching an "observation" from a data frame. The difference is when you change a data frame passed to a function, the change does not affect the original data frame. Only the function's local value is changed; ALL changes are forgotten when the function exits. If you change the data in an environment passed to a function, the change is permanent.)

```
library(mega2r)
ENV = read.Mega2DB("seqsimr.db", verbose = TRUE)
```

```
## int_table    34  3
## int_table    pId key value
##
## pedigree_table   20  8
## pedigree_table   pId Num EntryCnt    Name    PedPre  OriginalID  origped pedigree_link
##
## person_table 1380    11
## person_table pId UniqueID    OrigID  FamName PerPre  ID  Father  Mother  Sex pedigree_link   person_l
##
## pedigree_brkloop_table   20  8
## pedigree_brkloop_table   pId Num EntryCnt    Name    PedPre  OriginalID  origped pedigree_link
##
## person_brkloop_table 1380    11
## person_brkloop_table pId UniqueID    OrigID  FamName PerPre  ID  Father  Mother  Sex pedigree_link
##
## locus_table  1001    5
## locus_table  pId LocusName   Type    AlleleCnt   locus_link
##
## allele_table 2002    5
## allele_table pId AlleleName  Frequency   indexX  locus_link
##
## marker_table 1000    7
## marker_table pId MarkerName  pos_avg pos_female  pos_male    chromosome  locus_link
##
## markerscheme_table   1000    4
## markerscheme_table   pId key allele1 allele2
##
## map_table    2000    6
## map_table    pId marker  map position    pos_female  pos_male
##
## mapnames_table   2   6
## mapnames_table   pId map sex_averaged_map    male_sex_map    female_sex_map  name
##
## traitaff_table   1   4
## traitaff_table   pId ClassCnt    PenCnt  locus_link
##
## affectclass_table    1   9
## affectclass_table    pId MaleDef FemaleDef   AutoDef MalePen FemalePen   AutoPen locus_link  class_li
##
## phenotype_table  1380    4
## phenotype_table  pId person_link bytes   data
##
## genotype_table   1380    5
## genotype_table   pId person_link chr bytes   data
```

We need to make two observations that apply to all Mega2R functions: The first is that when verbose is set in the initial read.Mega2DB, the value will be remembered. It may be used by any subsequent function. If verbose is TRUE, Mega2R functions will possibly print out far more diagnostic information than you might need, especially on subsequent runs. The second is that all Mega2R functions that do not return an environment, need to have an environment supplied as an argument. There are two ways to accomplish this. If you assigned the result of read.Mega2DB to the variable seqsimr, then you could supply the value seqsimr to any Mega2R function as the named argument **envir**:

showMega2ENV(envir = seqsimr)

The second choice is a bit of a "hack" but it is very convenient. Every Mega2R function (that does not return an environment) has a named argument defined as

envir = ENV

as in

showMega2ENV = function(envir = ENV) { ... }

The code above, sets global variable **ENV** to the local variable, envir. And the argument evaluation mechanism of R will supports this usage; even though **ENV** is not known until runtime.

4.2.1 Back to more examples.

The ls function will show you all the variables in an environment. (You probably used it without arguments to show you the variables in the .GlobalEnv.) Type:

```
ls(ENV)
```

```
##  [1] "LocusCnt"               "MARKER_SCHEME"
##  [3] "PhenoCnt"               "affectclass_table"
##  [5] "allele_table"           "entrezGene"
##  [7] "fam"                    "int_table"
##  [9] "locus_allele_table"     "locus_table"
## [11] "map_table"              "mapnames_table"
## [13] "marker_table"           "markers"
## [15] "markerscheme_table"     "pedigree_brkloop_table"
## [17] "pedigree_table"         "person_brkloop_table"
## [19] "person_table"           "phenotype_table"
## [21] "refIndices"             "refRanges"
## [23] "traitaff_table"         "txdb"
## [25] "unified_genotype_table" "verbose"
```

4.2.2 A more informative overview of the data base can be seen by:

```
showMega2ENV()
```

```
## locus count:   1001; phenotype count:  1; compression: 2 bits
## marker count:  1000; sample count:  1380
##
## genetic and physical maps:
##   map name map number
## 1      Map          0
## 2       BP          1
##
## Phenotypes:
##   Index    Name      Type
## 1     1 default affection
##
##
## basic tables:
##                          rows cols
## affectclass_table           1    9
## allele_table             2002    5
## int_table                  34    3
## locus_table              1001    5
## map_table                2000    6
## mapnames_table              2    6
## marker_table             1000    8
## markerscheme_table       1000    4
## pedigree_brkloop_table     20    8
## pedigree_table             20    8
## person_brkloop_table     1380   11
## person_table             1380   11
## phenotype_table          1380    4
## traitaff_table              1    4
##
## derived tables:
##                          rows cols
## fam                      1380    8
## markers                  1000    5
## unified_genotype_table   1380    2
```

4.2.3 Use standard R commands to examine the created data frames

```
str(ENV$locus_table)
```

```
## 'data.frame':    1001 obs. of  5 variables:
##  $ pId       : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ LocusName : chr  "default" "snp1" "snp2" "snp3" ...
##  $ Type      : int  2 4 4 4 4 4 4 4 4 4 ...
##  $ AlleleCnt : int  2 2 2 2 2 2 2 2 2 2 ...
##  $ locus_link: int  0 1 2 3 4 5 6 7 8 9 ...
```

4.3 Running pedgene using the mega2pedgene R package

mega2pedgene is the exception that proves the rule. Rather than use the read.Mega2DB function described above and as used by all the other packages, we use init_pedgene. This function calls a utility function

created for read.Mega2DB. Then it creates, edits, and rewrites the **fam** data frame to purge persons with unknown case/control status. (**fam** merges data from the pedigree_table, person_table and phenotype_table as you might expect.) Finally, it calculates and stores some values that will be used repeatedly. viz:

```
library(mega2pedgene)
ENV = init_pedgene("seqsimr.db", verbose = T)
```

```
## int_table    34  3
## int_table    pId key value
##
## pedigree_table   20  8
## pedigree_table   pId Num EntryCnt    Name    PedPre  OriginalID  origped pedigree_link
##
## person_table 1380    11
## person_table pId UniqueID    OrigID  FamName PerPre  ID  Father  Mother  Sex pedigree_link   person_l
##
## pedigree_brkloop_table   20  8
## pedigree_brkloop_table   pId Num EntryCnt    Name    PedPre  OriginalID  origped pedigree_link
##
## person_brkloop_table 1380    11
## person_brkloop_table pId UniqueID    OrigID  FamName PerPre  ID  Father  Mother  Sex pedigree_link
##
## locus_table  1001    5
## locus_table  pId LocusName   Type    AlleleCnt   locus_link
##
## allele_table 2002    5
## allele_table pId AlleleName  Frequency   indexX  locus_link
##
## marker_table 1000    7
## marker_table pId MarkerName  pos_avg pos_female  pos_male    chromosome  locus_link
##
## markerscheme_table   1000    4
## markerscheme_table   pId key allele1 allele2
##
## map_table    2000    6
## map_table    pId marker  map position    pos_female  pos_male
##
## mapnames_table   2   6
## mapnames_table   pId map sex_averaged_map    male_sex_map    female_sex_map  name
##
## traitaff_table   1   4
## traitaff_table   pId ClassCnt    PenCnt  locus_link
##
## affectclass_table    1   9
## affectclass_table    pId MaleDef FemaleDef   AutoDef MalePen FemalePen   AutoPen locus_link  class_li
##
## phenotype_table  1380    4
## phenotype_table  pId person_link bytes   data
##
## genotype_table   1380    5
## genotype_table   pId person_link chr bytes   data
```

Mega2R has an internal default list of the chromosome and base pair ranges for a number of gene transcripts. These transcripts come from the UCSC Genome Browser reference assembly GRCH37. The list was further

modified to eliminate multiple records of the same gene with the exact same transcript start and transcript end. This constitutes about 30,000 records. The function run_pedgene examines the first 100 transcripts and prints the results. The first 100 transcripts identifies one gene with several markers:

```
run_pedgene()
## No markers in range:  chr20 between 62737182 and 62738184
## No markers in range:  chr6 between 26924771 and 26991753
## No markers in range:  chr7 between 150725509 and 150744869
## No markers in range:  chr7 between 102993176 and 103086624
## No markers in range:  chr5 between 180581942 and 180582890
## No markers in range:  chr2 between 28974613 and 29025806
## No markers in range:  chr11 between 129872518 and 129875381
## No markers in range:  chr14 between 94463615 and 94473898
## No markers in range:  chr12 between 113659259 and 113736389
## No markers in range:  chr1 between 1309109 and 1310818
## No markers in range:  chr2 between 216807313 and 216878346
## No markers in range:  chr2 between 37311593 and 37323738
## No markers in range:  chr19 between 2252249 and 2256422
## No markers in range:  chr18 between 44057216 and 44236996
## No markers in range:  chr6 between 52842745 and 52860178
## No markers in range:  chr10 between 64893006 and 64914786
## No markers in range:  chr14 between 24708850 and 24711880
## No markers in range:  chr15 between 44119111 and 44160617
## No markers in range:  chr6 between 41126243 and 41130924
## No markers in range:  chr19 between 41892275 and 41903256
## No markers in range:  chr20 between 16729002 and 16732809
## No markers in range:  chr17 between 38632079 and 38657854
## No markers in range:  chr8 between 144680073 and 144682485
## No markers in range:  chr8 between 38268655 and 38326352
## No markers in range:  chr1 between 32712817 and 32714461
## No markers in range:  chr8 between 82711817 and 82754521
## No markers in range:  chr13 between 31480311 and 31499709
## No markers in range:  chr3 between 180701497 and 180707562
## No markers in range:  chr22 between 24647588 and 24661492
## No markers in range:  chr1 between 111023387 and 111033891
## No markers in range:  chr9 between 139702373 and 139728928
## No markers in range:  chr3 between 51991469 and 52001482
## No markers in range:  chr11 between 57529233 and 57586652
## No markers in range:  chr16 between 89803958 and 89883065
## No markers in range:  chr12 between 109015679 and 109025854
## No markers in range:  chr11 between 49075265 and 49080664
## No markers in range:  chr2 between 130831107 and 130886795
## No markers in range:  chr17 between 38171613 and 38174066
## No markers in range:  chr11 between 102217912 and 102249401
## No markers in range:  chr11 between 102188180 and 102210135
## No markers in range:  chr5 between 98190907 and 98262238
## No markers in range:  chr4 between 174252526 and 174255595
## No markers in range:  chr17 between 40336077 and 40337470
## No markers in range:  chr17 between 80332200 and 80333370
## No markers in range:  chr6 between 29691116 and 29694303
## No markers in range:  chr12 between 3068477 and 3149842
## No markers in range:  chr1 between 228294379 and 228297013
## No markers in range:  chr11 between 34654010 and 34684834
## No markers in range:  chr1 between 153631129 and 153634328
```

```
## No markers in range:  chr9 between 34653893 and 34661898
## No markers in range:  chr1 between 203185206 and 203198860
## No markers in range:  chr14 between 57046510 and 57116232
## No markers in range:  chr7 between 111366163 and 111846462
## CEP104 snp24037 520 646 214
## CEP104 snp24039 940 386 54
## CEP104 snp24041 1377 3 0
## CEP104 snp24048 860 460 60
## CEP104 snp24494 789 501 90
## CEP104 snp24499 891 442 47
## CEP104 snp24506 891 442 47
## CEP104 snp24507 789 501 90
##    chr   gene nvariants   start     end sKernel_BT pKernel_BT sBurden_BT
## 1 chr1 CEP104         8 3728644 3773797   31.96998  0.6297541 -0.6496837
##   pBurden_BT sKernel_MB pKernel_MB sBurden_MB pBurden_MB sKernel_UW
## 1  0.5158965   1184.995  0.5138866  -1.209563  0.2264468   222.8737
##   pKernel_UW sBurden_UW pBurden_UW
## 1  0.4111136  -1.169488   0.242207
## No markers in range:  chr4 between 72204769 and 72437804
## No markers in range:  chr7 between 41728600 and 41742706
## No markers in range:  chr5 between 50678957 and 50690563
## No markers in range:  chr2 between 178257470 and 178408564
## No markers in range:  chr11 between 129245880 and 129322174
## No markers in range:  chr7 between 158649268 and 158738883
## No markers in range:  chr8 between 74903563 and 74941307
## No markers in range:  chr3 between 150644353 and 150690786
## No markers in range:  chr16 between 30097114 and 30103205
## No markers in range:  chr8 between 133584200 and 133687863
## No markers in range:  chr7 between 128594233 and 128695227
## No markers in range:  chr8 between 139142265 and 139509065
## No markers in range:  chr22 between 21771692 and 21805750
## No markers in range:  chr6 between 149639435 and 149732747
## No markers in range:  chr22 between 51205919 and 51222087
## No markers in range:  chr12 between 104609556 and 104744085
## No markers in range:  chr22 between 51007289 and 51016506
## No markers in range:  chr4 between 109663201 and 109684235
## No markers in range:  chr11 between 5710816 and 5732093
## No markers in range:  chr19 between 36505561 and 36523573
## No markers in range:  chr10 between 114710008 and 114927436
## No markers in range:  chr1 between 62920396 and 63154039
## No markers in range:  chr16 between 610421 and 615529
## No markers in range:  chr11 between 65479472 and 65487077
## No markers in range:  chr10 between 27484142 and 27529808
## No markers in range:  chr7 between 129847703 and 129856684
## No markers in range:  chr9 between 19507449 and 19787017
## No markers in range:  chr17 between 76792964 and 76836969
## No markers in range:  chr10 between 131862161 and 131909081
## No markers in range:  chr5 between 80533383 and 80597388
## No markers in range:  chr13 between 107028910 and 107030142
## No markers in range:  chr7 between 130565750 and 130598069
## No markers in range:  chr20 between 23105704 and 23113273
## No markers in range:  chr21 between 34106209 and 34144169
## No markers in range:  chr1 between 104615644 and 104619693
```

```
## No markers in range:  chr3 between 195869506 and 195887761
## No markers in range:  chr14 between 90921573 and 90925249
## No markers in range:  chr16 between 34980922 and 34990995
## No markers in range:  chr12 between 69004618 and 69054385
## No markers in range:  chr6 between 3118925 and 3153432
## No markers in range:  chr15 between 28982728 and 29003508
## No markers in range:  chr1 between 245133630 and 245288530
## No markers in range:  chr1 between 245133283 and 245288530
## No markers in range:  chr17 between 38183794 and 38183898
## No markers in range:  chr5 between 9548938 and 9549026
## No markers in range:  chr19 between 10220424 and 10220516
```

You will see many reports of "No markers in range", but occasionally you will see a listing of a gene name, markers, and count of 1/1, 1/2 and 2/2 genotypes, viz.

CEP104 snp24037 520 646 214
CEP104 snp24039 940 386 54
CEP104 snp24041 1377 3 0
CEP104 snp24048 860 460 60
CEP104 snp24494 789 501 90
CEP104 snp24499 891 442 47
CEP104 snp24506 891 442 47
CEP104 snp24507 789 501 90

The genotypes for these markers, along with the markers info are fed to the call back function DOpedgene. DOpedgene converts the genotype nucleotides to the count of the minor allele and then runs pedgene for several different marker weights. The results are automatically stored in a data frame with "observations": prefix of chromosome, gene, number of markers and base pair range followed by Pedgene data: kernel and burden, value and p-values, four values for each of three weightings of the markers. It is printed as well when *verbose* is TRUE, viz.

chr gene nvariants start end sKernel_BT pKernel_BT sBurden_BT
1 chr1 CEP104 8 3728644 3773797 31.96998 0.6297541 -0.6496837
pBurden_BT sKernel_MB pKernel_MB sBurden_MB pBurden_MB sKernel_UW pKernel_UW
1 0.5158965 1184.995 0.5138866 -1.209563 0.2264468 222.8737 0.4111136
sBurden_UW pBurden_UW
1 -1.169488 0.242207

It would be better to run pedgene on all the default transcript entries. So type:

```
# we will skip this line for the Rmd document production because it takes too
# long
applyFnToRanges(DOpedgene, ENV$refRanges, ENV$refIndices, envir = ENV)
```

If you run the above test, you will see that gene DISP1 and KIF26B have some p-values less than 0.01 and AK5 and STL7 less than 0.03.

Alternatively, you may try searching by Gene. Here the default transcript database is TxDb.Hsapiens.UCSC.hg19.knownGene from Bioconductor. Of course you can change it. Type:

setAnnotations("txdb", "genedb")

where "txdb" is a string that is the name of transcript database that was fetched from Bioconductor, and similarly "genedb" is the name of a Bioconductor database that maps a gene id from the transcript to an entrez gene id.

```
applyFnToGenes(DOpedgene, genes_arg = c("DISP1", "KIF26B", "AK5", "ST7L"), envir = ENV)
## 'select()' returned 1:1 mapping between keys and columns
## 'select()' returned 1:many mapping between keys and columns
## No markers in range:  chr1 between 77747662 and 77780745
## AK5 snp80127 962 375 43
## AK5 snp80131 1095 272 13
## AK5 snp80142 1303 77 0
## AK5 snp80148 1345 35 0
##   chr gene nvariants    start      end sKernel_BT pKernel_BT sBurden_BT
## 2   1  AK5         4 77747662 78025654   2593.945   0.228311  -1.400928
##   pBurden_BT sKernel_MB pKernel_MB sBurden_MB pBurden_MB sKernel_UW
## 2  0.1612355   1899.249 0.06206979  -2.263002 0.02363554   175.4195
##   pKernel_UW sBurden_UW pBurden_UW
## 2   0.051013  -2.290307  0.0220035
## AK5 snp80127 962 375 43
## AK5 snp80131 1095 272 13
## AK5 snp80142 1303 77 0
## AK5 snp80148 1345 35 0
##   chr gene nvariants    start      end sKernel_BT pKernel_BT sBurden_BT
## 3   1  AK5         4 77748287 78025654   2593.945   0.228311  -1.400928
##   pBurden_BT sKernel_MB pKernel_MB sBurden_MB pBurden_MB sKernel_UW
## 3  0.1612355   1899.249 0.06206979  -2.263002 0.02363554   175.4195
##   pKernel_UW sBurden_UW pBurden_UW
## 3   0.051013  -2.290307  0.0220035
## ST7L snp144498 795 510 75
## ST7L snp144501 980 370 30
##   chr gene nvariants     start       end sKernel_BT pKernel_BT sBurden_BT
## 4   1 ST7L         2 113066141 113153625    8.55093  0.1867296  -1.085732
##   pBurden_BT sKernel_MB pKernel_MB sBurden_MB pBurden_MB sKernel_UW
## 4  0.2775977   1387.121 0.03117574  0.6917602  0.4890879   235.0836
##   pKernel_UW sBurden_UW pBurden_UW
## 4 0.02377278  0.8952679  0.3706439
## ST7L snp144498 795 510 75
## ST7L snp144501 980 370 30
##   chr gene nvariants     start       end sKernel_BT pKernel_BT sBurden_BT
## 5   1 ST7L         2 113066141 113161550    8.55093  0.1867296  -1.085732
##   pBurden_BT sKernel_MB pKernel_MB sBurden_MB pBurden_MB sKernel_UW
## 5  0.2775977   1387.121 0.03117574  0.6917602  0.4890879   235.0836
##   pKernel_UW sBurden_UW pBurden_UW
## 5 0.02377278  0.8952679  0.3706439
## ST7L snp144498 795 510 75
## ST7L snp144501 980 370 30
##   chr gene nvariants     start       end sKernel_BT pKernel_BT sBurden_BT
## 6   1 ST7L         2 113066141 113161761    8.55093  0.1867296  -1.085732
##   pBurden_BT sKernel_MB pKernel_MB sBurden_MB pBurden_MB sKernel_UW
## 6  0.2775977   1387.121 0.03117574  0.6917602  0.4890879   235.0836
##   pKernel_UW sBurden_UW pBurden_UW
## 6 0.02377278  0.8952679  0.3706439
## ST7L snp144498 795 510 75
## ST7L snp144501 980 370 30
```

```
##   chr gene nvariants     start       end sKernel_BT pKernel_BT sBurden_BT
## 7   1 ST7L         2 113066141 113161761    8.55093  0.1867296  -1.085732
##   pBurden_BT sKernel_MB pKernel_MB sBurden_MB pBurden_MB sKernel_UW
## 7  0.2775977   1387.121 0.03117574  0.6917602  0.4890879   235.0836
##   pKernel_UW sBurden_UW pBurden_UW
## 7 0.02377278  0.8952679  0.3706439
## ST7L snp144498 795 510 75
## ST7L snp144501 980 370 30
##   chr gene nvariants     start       end sKernel_BT pKernel_BT sBurden_BT
## 8   1 ST7L         2 113066141 113162040    8.55093  0.1867296  -1.085732
##   pBurden_BT sKernel_MB pKernel_MB sBurden_MB pBurden_MB sKernel_UW
## 8  0.2775977   1387.121 0.03117574  0.6917602  0.4890879   235.0836
##   pKernel_UW sBurden_UW pBurden_UW
## 8 0.02377278  0.8952679  0.3706439
## ST7L snp144498 795 510 75
## ST7L snp144501 980 370 30
##   chr gene nvariants     start       end sKernel_BT pKernel_BT sBurden_BT
## 9   1 ST7L         2 113066141 113162040    8.55093  0.1867296  -1.085732
##   pBurden_BT sKernel_MB pKernel_MB sBurden_MB pBurden_MB sKernel_UW
## 9  0.2775977   1387.121 0.03117574  0.6917602  0.4890879   235.0836
##   pKernel_UW sBurden_UW pBurden_UW
## 9 0.02377278  0.8952679  0.3706439
## ST7L snp144498 795 510 75
## ST7L snp144501 980 370 30
##    chr gene nvariants     start       end sKernel_BT pKernel_BT sBurden_BT
## 10   1 ST7L         2 113066141 113162040    8.55093  0.1867296  -1.085732
##    pBurden_BT sKernel_MB pKernel_MB sBurden_MB pBurden_MB sKernel_UW
## 10  0.2775977   1387.121 0.03117574  0.6917602  0.4890879   235.0836
##    pKernel_UW sBurden_UW pBurden_UW
## 10 0.02377278  0.8952679  0.3706439
## ST7L snp144498 795 510 75
## ST7L snp144501 980 370 30
##    chr gene nvariants     start       end sKernel_BT pKernel_BT sBurden_BT
## 11   1 ST7L         2 113066141 113162405    8.55093  0.1867296  -1.085732
##    pBurden_BT sKernel_MB pKernel_MB sBurden_MB pBurden_MB sKernel_UW
## 11  0.2775977   1387.121 0.03117574  0.6917602  0.4890879   235.0836
##    pKernel_UW sBurden_UW pBurden_UW
## 11 0.02377278  0.8952679  0.3706439
## ST7L snp144498 795 510 75
## ST7L snp144501 980 370 30
##    chr gene nvariants     start       end sKernel_BT pKernel_BT sBurden_BT
## 12   1 ST7L         2 113084413 113160839    8.55093  0.1867296  -1.085732
##    pBurden_BT sKernel_MB pKernel_MB sBurden_MB pBurden_MB sKernel_UW
## 12  0.2775977   1387.121 0.03117574  0.6917602  0.4890879   235.0836
##    pKernel_UW sBurden_UW pBurden_UW
## 12 0.02377278  0.8952679  0.3706439
## ST7L snp144498 795 510 75
## ST7L snp144501 980 370 30
##    chr gene nvariants     start       end sKernel_BT pKernel_BT sBurden_BT
## 13   1 ST7L         2 113084413 113161761    8.55093  0.1867296  -1.085732
##    pBurden_BT sKernel_MB pKernel_MB sBurden_MB pBurden_MB sKernel_UW
## 13  0.2775977   1387.121 0.03117574  0.6917602  0.4890879   235.0836
##    pKernel_UW sBurden_UW pBurden_UW
```

```
## 13 0.02377278  0.8952679  0.3706439
## ST7L snp144498 795 510 75
## ST7L snp144501 980 370 30
##    chr gene nvariants     start       end sKernel_BT pKernel_BT sBurden_BT
## 14   1 ST7L         2 113084413 113162040    8.55093  0.1867296  -1.085732
##    pBurden_BT sKernel_MB pKernel_MB sBurden_MB pBurden_MB sKernel_UW
## 14  0.2775977   1387.121 0.03117574  0.6917602  0.4890879   235.0836
##    pKernel_UW sBurden_UW pBurden_UW
## 14 0.02377278  0.8952679  0.3706439
## ST7L snp144498 795 510 75
## ST7L snp144501 980 370 30
##    chr gene nvariants     start       end sKernel_BT pKernel_BT sBurden_BT
## 15   1 ST7L         2 113084573 113161761    8.55093  0.1867296  -1.085732
##    pBurden_BT sKernel_MB pKernel_MB sBurden_MB pBurden_MB sKernel_UW
## 15  0.2775977   1387.121 0.03117574  0.6917602  0.4890879   235.0836
##    pKernel_UW sBurden_UW pBurden_UW
## 15 0.02377278  0.8952679  0.3706439
## ST7L snp144498 795 510 75
## ST7L snp144501 980 370 30
##    chr gene nvariants     start       end sKernel_BT pKernel_BT sBurden_BT
## 16   1 ST7L         2 113093223 113161761    8.55093  0.1867296  -1.085732
##    pBurden_BT sKernel_MB pKernel_MB sBurden_MB pBurden_MB sKernel_UW
## 16  0.2775977   1387.121 0.03117574  0.6917602  0.4890879   235.0836
##    pKernel_UW sBurden_UW pBurden_UW
## 16 0.02377278  0.8952679  0.3706439
## KIF26B snp358996 1363 17 0
## KIF26B snp358997 860 445 75
## KIF26B snp358999 1215 160 5
## KIF26B snp359000 1359 21 0
## KIF26B snp359211 1185 191 4
## KIF26B snp359213 1267 112 1
## KIF26B snp359214 1194 182 4
## KIF26B snp359220 1205 171 4
##    chr   gene nvariants     start       end sKernel_BT  pKernel_BT
## 17   1 KIF26B         8 245318287 245809683   12582.46 0.004587158
##    sBurden_BT pBurden_BT sKernel_MB  pKernel_MB sBurden_MB pBurden_MB
## 17   2.279102 0.02266102    5961.57 0.009231123   2.024773 0.04289063
##    sKernel_UW pKernel_UW sBurden_UW pBurden_UW
## 17   251.7185 0.04775414   1.465863  0.1426855
## KIF26B snp358996 1363 17 0
## KIF26B snp358997 860 445 75
## KIF26B snp358999 1215 160 5
## KIF26B snp359000 1359 21 0
## KIF26B snp359211 1185 191 4
## KIF26B snp359213 1267 112 1
## KIF26B snp359214 1194 182 4
## KIF26B snp359220 1205 171 4
##    chr   gene nvariants     start       end sKernel_BT  pKernel_BT
## 18   1 KIF26B         8 245318287 245866428   12582.46 0.004587158
##    sBurden_BT pBurden_BT sKernel_MB  pKernel_MB sBurden_MB pBurden_MB
## 18   2.279102 0.02266102    5961.57 0.009231123   2.024773 0.04289063
##    sKernel_UW pKernel_UW sBurden_UW pBurden_UW
## 18   251.7185 0.04775414   1.465863  0.1426855
```

```
## No markers in range:  chr1 between 245516980 and 245535019
## No markers in range:  chr1 between 245674304 and 245852109
## No markers in range:  chr1 between 245809394 and 245852109
## DISP1 snp332780 1375 5 0
## DISP1 snp332781 675 559 146
## DISP1 snp332783 1365 15 0
## DISP1 snp332784 898 427 55
##    chr  gene nvariants     start       end sKernel_BT  pKernel_BT
## 19   1 DISP1         4 222988431 223179337    4559.34 0.004224181
##    sBurden_BT pBurden_BT sKernel_MB  pKernel_MB sBurden_MB  pBurden_MB
## 19    1.44652  0.1480315   5410.868 0.000253584   2.916462 0.003540261
##    sKernel_UW pKernel_UW sBurden_UW pBurden_UW
## 19    277.508 0.04407319   2.094983 0.03617254
```

NOTE: You will observe that for several genes there are a few transcripts with the same start values and the same end values.

4.4 Writing a VCF File using the mega2vcf R package

First create the directory "vcfr"; so we can keep all the generated data in one place and then load the mega2vcf library.

The Mega2VCF function takes an argument which is the prefix used on all the files it generates. We have chosen to make that a path that includes a directory. Mega2VCF is explained in greater depth in the Implementation Section. The Mega2VCF function assumes the environment is contained in the global variable ENV. If this is not true supply an additional argument "envir" set to the environment you wish to use.

Note, that Mega2VCF will create the expected .vcf file as well as a .fam file (we do linkage), a .phe file (phenotypes) and a few others. These files are listed below.

Creating the VCF file and related files is accomplished by typing:

```
if (!dir.exists("vcfr")) dir.create("vcfr")
library(mega2vcf)
Mega2VCF("vcfr/vcf.01", ENV$markers[ENV$markers$chromosome == 1, ])
```

```
## .
```

```
list.files("vcfr")
```

```
## [1] "vcf.01.fam"  "vcf.01.freq" "vcf.01.map"  "vcf.01.pen"  "vcf.01.phe"
## [6] "vcf.01.vcf"
```

Read the package code to see how these files are created from the data frames.

4.5 Creating a GenABEL gwaa.data-class object using the mega2genabel R package

GenABEL (http://www.genabel.org/) can process PLINK .tped/.tfam files with the function *convert.snp.tped* to create a gwaa.data-class object. Currently, we use this mechanism to convert our data frames to a gwaa.data-class object. The argument to Mega2GenABEL is a file name prefix that will be used to name the

.tped/.fam files that are created as an intermediate to be read by GenABEL (https://cran.r-project.org/web/packages/GenABEL/index.html) to produce a GenABEL "raw" file. Then the "raw" file and a generated phenotype file are processed by *load.gwaa.data* to yield a gwaa.data-class object. (See the Implementation Section.)

This task is analogous to the previous exercise. Type:

```
library(mega2genabel)
seqsimgwaa = Mega2GenABEL("SEQ")
```

```
## .

## Reading individual ids from file 'SEQ.tfam' ...
## ... done.  Read 1380 individual ids from file 'SEQ.tfam'
## Reading genotypes from file 'SEQ.tped' ...
## ...done.  Read 1000 SNPs from file 'SEQ.tped'
## Writing to file 'SEQtped.raw' ...
## ... done.
## ids loaded...
## marker names loaded...
## chromosome data loaded...
## map data loaded...
## allele coding data loaded...
## strand data loaded...
## genotype data loaded...
## snp.data object created...
## assignment of gwaa.data object FORCED; X-errors were not checked!
```

```
list.files(pattern = "SEQ.*")
```

```
## [1] "SEQ.phe"     "SEQ.tfam"    "SEQ.tped"    "SEQtped.raw"
```

```
str(seqsimgwaa)
```

```
## Formal class 'gwaa.data' [package "GenABEL"] with 2 slots
##   ..@ phdata:'data.frame':   1380 obs. of  3 variables:
##   .. ..$ id     : chr [1:1380] "1SAP039_H05-0107" "1SAP039_H05-0106" "1SAP039_SAP039F14" "1SAP039_SA
##   .. ..$ sex    : int [1:1380] 0 1 1 1 1 0 0 0 0 0 ...
##   .. ..$ default: int [1:1380] 1 1 1 1 1 1 1 1 1 1 ...
##   ..@ gtdata:Formal class 'snp.data' [package "GenABEL"] with 11 slots
##   .. .. ..@ nbytes    : num 345
##   .. .. ..@ nids      : int 1380
##   .. .. ..@ nsnps     : int 1000
##   .. .. ..@ idnames   : chr [1:1380] "1SAP039_H05-0107" "1SAP039_H05-0106" "1SAP039_SAP039F14" "1SAP
##   .. .. ..@ snpnames  : chr [1:1000] "snp1" "snp2" "snp3" "snp4" ...
##   .. .. ..@ chromosome: Factor w/ 1 level "1": 1 1 1 1 1 1 1 1 1 1 ...
##   .. .. .. ..- attr(*, "names")= chr [1:1000] "snp1" "snp2" "snp3" "snp4" ...
##   .. .. ..@ map       : Named num [1:1000] 2 3 4 5 201 ...
##   .. .. .. ..- attr(*, "names")= chr [1:1000] "snp1" "snp2" "snp3" "snp4" ...
##   .. .. ..@ coding    :Formal class 'snp.coding' [package "GenABEL"] with 1 slot
##   .. .. .. .. ..@ .Data: raw [1:1000] 01 01 01 01 ...
##   .. .. ..@ strand    :Formal class 'snp.strand' [package "GenABEL"] with 1 slot
```

```
##   .. .. .. .. ..@ .Data: raw [1:1000] 00 00 00 00 ...
##   .. .. ..@ male      : Named int [1:1380] 0 1 1 1 1 0 0 0 0 0 ...
##   .. .. .. ..- attr(*, "names")= chr [1:1380] "1SAP039_H05-0107" "1SAP039_H05-0106" "1SAP039_SAP039F
##   .. .. ..@ gtps      :Formal class 'snp.mx' [package "GenABEL"] with 1 slot
##   .. .. .. .. ..@ .Data: raw [1:345, 1:1000] 59 55 55 55 ...
```

You can use any of the GenABEL provided functions on your data, seqsimgwaa.

Note: There are several data formats that Mega2 can read and transform into a database that are not directly acceptable to GenABEL. Any M1ega2 database that can be read into R, can be transformed to GenABEL following this example with seqsimr.db.

5 Regression

To verify that the R packages described above have worked correctly, we can compare their output files to those created by Mega2 itself, to verify that they are, as expected, identical. This type of testing is known as regression testing.

5.1 VCF regression

It is time to use the MEGA2.BATCH.vcf file from the mega2rtutorial. This will use the C++ Mega2 program to populate the vcf directory with the same set of files that the R program created in the vcfr directory. At the Unix command prompt, type:

```
mkdir vcf
mega2 MEGA2.BATCH.vcf
```

The abbreviated MEGA2.BATCH.vcf file is below (notice that there are no INPUT FILES, just a database file):

Input_Database_Mode=2
Align_Strand_Input=no
Output_Path=vcf
Input_Untyped_Ped_Option=2
Input_Do_Error_Sim=no
AlleleFreq_SquaredDev=999999999.000000
Analysis_Option=VCF
Value_Missing_Quant_On_Output=-9
Value_Missing_Affect_On_Output=-9
DBfile_name=seqsimr.db
Chromosome_Single=1
Traits_Combine=1 2 e
file_name_stem=vcf
human_genome_build=B37
VCF_output_file_type=1
VCF_Allele_Order=Original_Order
Default_Outfile_Names=yes

You should compare the two directories: vcf and vcfr. Please add the -w flag to the diff command. This causes the comparison to ignore differences in white space. You may may notice that expressions that contain dates that are different between the two directories. Type:

```
diff -wrs vcf vcfr
```

```
## Files vcf/vcf.01.fam and vcfr/vcf.01.fam are identical
## Files vcf/vcf.01.freq and vcfr/vcf.01.freq are identical
## Files vcf/vcf.01.map and vcfr/vcf.01.map are identical
## Files vcf/vcf.01.pen and vcfr/vcf.01.pen are identical
## Files vcf/vcf.01.phe and vcfr/vcf.01.phe are identical
## Files vcf/vcf.01.vcf and vcfr/vcf.01.vcf are identical
```

5.2 GenABEL regression

This test is a bit more complicated. First, we load GenABEL and access its data:

```
library(GenABEL)
data(srdta)
```

Then we load the mega2genabel library and dump the GenABEL data as PLINK .ped/.map/.phe files

```
library(mega2genabel)
dmpPed()
```

Then we switch to the Unix command line and run:

```
mega2 MEGA2.BATCH.srdta
```

5.2.1 (abbreviated contents of MEGA2.BATCH.srdata file

Input_Database_Mode=1
Input_Format_Type=4
Input_Pedigree_File=srdta.ped
Input_PLINK_Map_File=srdta.map
Input_Phenotype_File=srdta.phe
Output_Path=.
Input_Path=.
PLINK_Args= –missing-phenotype -9 –trait default
Input_Untyped_Ped_Option=2
Input_Do_Error_Sim=no
AlleleFreq_SquaredDev=999999999.000000
Value_Marker_Compression=1
Analysis_Option=Dump
Value_Missing_Quant_On_Input=-9.000000
Value_Missing_Affect_On_Input=-9
Count_Genotypes=4
Count_Halftyped=no
Value_Genetic_Distance_Index=0
Value_Genetic_Distance_SexTypeMap=0
Value_Base_Pair_Position_Index=1
Default_Reset_Invalid=no
DBfile_name=srdta.db
Default_Outfile_Names=yes

to produce the Mega2 database srdta.db.

Then we go back into R and type:

```
ENV = read.Mega2DB("srdta.db")
mega = Mega2GenABEL("SRD")
```

```
## Reading individual ids from file 'SRD.tfam' ...
## ... done.  Read 2500 individual ids from file 'SRD.tfam'
## Reading genotypes from file 'SRD.tped' ...
## ...done.  Read 833 SNPs from file 'SRD.tped'
## Writing to file 'SRDtped.raw' ...
## ... done.
## ids loaded...
## marker names loaded...
## chromosome data loaded...
## map data loaded...
## allele coding data loaded...
## strand data loaded...
## genotype data loaded...
## snp.data object created...
## assignment of gwaa.data object FORCED; X-errors were not checked!
```

```
list.files(pattern = "SRD.*")
```

```
## [1] "SRD.phe"     "SRD.tfam"    "SRD.tped"    "SRDtped.raw"
```

mega and *srdta* should be the same gwaa.data-class object. You can compare them however you prefer. But, Mega2GenABELtst can be used to compare the phenotype and genotype data, viz. in R type:

```
Mega2GenABELtst()
```

```
## all(mega_@phdata$sex == gwaa_@phdata$sex)[1] TRUE
## all(mega_@phdata$age == gwaa_@phdata$age)[1] TRUE
## all(mega_@phdata$qt1 == gwaa_@phdata$qt1)[1] TRUE
## all(mega_@phdata$qt2 == gwaa_@phdata$qt2)[1] TRUE
## all(mega_@phdata$qt3 == gwaa_@phdata$qt3)[1] TRUE
## all(mega_@phdata$bt == gwaa_@phdata$bt)[1] TRUE
## all(mega_@phdata$default == gwaa_@phdata$default)[1] TRUE
## all(mega_@gtdata@nids == gwaa_@gtdata@nids)[1] TRUE
## all(mega_@gtdata@nsnps == gwaa_@gtdata@nsnps)[1] TRUE
## all(mega_@gtdata@nbytes == gwaa_@gtdata@nbytes)[1] TRUE
## all(mega_@gtdata@snpnames == gwaa_@gtdata@snpnames)[1] TRUE
## all(mega_@gtdata@chromosome == gwaa_@gtdata@chromosome)[1] TRUE
## all(mega_@gtdata@map == gwaa_@gtdata@map)[1] TRUE
## all(mega_@gtdata@male == gwaa_@gtdata@male)[1] TRUE
## all(mega_@gtdata == gwaa_@gtdata)[1] TRUE
```

6 Next steps

If you are left with questions as to how you can use Mega2R for your own research, you can:

1. Read our paper "The Mega2R suite of R packages: tools for accessing and processing common genetic data formats in R", which extends this document by providing implementation details.

2. Read the source for the packages if you understand R; the code is not particularly subtle.

3. Write to the Mega2 discussion group: https://groups.google.com/forum/#!forum/mega2-users